

Complete DSA Algorithms Guide

12 Essential Algorithms with Methodology, Pseudocode & User-Input Java Code

1. Euclid's Algorithm (HCF / GCD)

1.1 Brute Force Approach

Methodology

- Start from the smaller of the two numbers
- Check every number downward until both divide evenly
- First common divisor found is the HCF

Pseudocode

```
FUNCTION bruteForceHCF(a, b):  
    gcf = -1  
    FOR i = min(a, b) DOWNTO 1:  
        IF a % i == 0 AND b % i == 0:  
            gcf = i  
            BREAK  
    RETURN gcf
```

 $O(\min(a,b))$

Source Code

```
package com.basics;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        int gcf = -1;

        for (int i = Math.min(a, b); i >= 1; i--) {
            if (a % i == 0 && b % i == 0) {
                gcf = i;
                break;
            }
        }

        System.out.println("HCF = " + gcf);
        sc.close();
    }
}
```

1.2 Subtraction Method

Methodology

- Repeatedly subtract the smaller number from the larger
- Continue until both numbers become equal
- The common value is the HCF

Pseudocode

```
FUNCTION subtractionHCF(a, b):  
    WHILE a != b:  
        IF a > b: a = a - b  
        ELSE: b = b - a  
    RETURN a
```

 $O(\max(a,b))$

Source Code

```
package com.basics;  
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter first number: ");  
        int a = sc.nextInt();  
  
        System.out.print("Enter second number: ");  
        int b = sc.nextInt();  
  
        while (a != b) {  
            if (a > b) a = a - b;  
            else b = b - a;  
        }  
  
        System.out.println("HCF = " + a);  
        sc.close();  
    }  
}
```

1.3 Euclidean Algorithm (Modulus Method)

Methodology

- Replace (a, b) with (b, a % b)
- Repeat until b becomes 0
- When b = 0, a contains the HCF

Pseudocode

```
FUNCTION euclideanHCF(a, b):  
    WHILE b != 0:  
        temp = a % b  
        a = b  
        b = temp  
    RETURN a
```

 $O(\log(\min(a,b)))$

Source Code

```
package com.basics;  
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter first number: ");  
        int a = sc.nextInt();  
  
        System.out.print("Enter second number: ");  
        int b = sc.nextInt();  
  
        while (b != 0) {  
            int temp = a % b;  
            a = b;  
            b = temp;  
        }  
  
        System.out.println("HCF = " + a);  
        sc.close();  
    }  
}
```

2. Karatsuba Algorithm

Methodology

- Divide each number into two halves: high and low
- Recursively compute three products: ac , bd , $(a+b)(c+d)$
- Combine using: $ac \times 10^{(2m)} + ((a+b)(c+d) - ac - bd) \times 10^m + bd$

Pseudocode

```

FUNCTION countDigits(n):
    IF n == 0: RETURN 1
    count = 0
    WHILE n > 0:
        count++
        n = n / 10
    RETURN count

FUNCTION karatsuba(x, y):
    IF x < 10 OR y < 10:
        RETURN x * y
    n = max(countDigits(x), countDigits(y))
    m = n / 2
    tenpowM = 10^m
    a = x / tenpowM
    b = x % tenpowM
    c = y / tenpowM
    d = y % tenpowM
    ac = karatsuba(a, c)
    bd = karatsuba(b, d)
    midExp = karatsuba(a+b, c+d) - ac - bd
    RETURN ac * tenpowM * tenpowM + midExp * tenpowM + bd
  
```

 $O(n^{1.585})$

Source Code

```
package com.basics;

import java.util.Scanner;

class Main {

    static long karatsuba(long x, long y) {
        if (x < 10 || y < 10) {
            return x * y;
        }

        int n = Math.max(countDigits(x), countDigits(y));
        int m = n / 2;
        long tenpowM = (long) Math.pow(10, m);

        long a = x / tenpowM;
        long b = x % tenpowM;
        long c = y / tenpowM;
        long d = y % tenpowM;

        long ac = karatsuba(a, c);
        long bd = karatsuba(b, d);
        long midExp = karatsuba(a + b, c + d) - ac - bd;

        long res = ac * tenpowM * tenpowM + midExp * tenpowM + bd;
        return res;
    }

    static int countDigits(long n) {
        if (n == 0) return 1;
        int count = 0;
        while (n > 0) {
            count++;
            n /= 10;
        }
        return count;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        long x = sc.nextLong();

        System.out.print("Enter second number: ");
        long y = sc.nextLong();

        long result = karatsuba(x, y);
```

```
        System.out.println("Product = " + result);

        sc.close();
    }
}
```

3. Longest Sequence of 1s After Flipping Bits

3.1 Brute Force (Flip 1 bit)

Methodology

- For each starting position, extend window
- Count zeros; if more than 1 zero, break
- Track maximum window size

Pseudocode

```
FUNCTION bruteForce(ar):
    max = 0
    FOR i = 0 TO n-1:
        zeroCount = 0
        FOR j = i TO n-1:
            IF ar[j] == 0: zeroCount++
            IF zeroCount > 1: BREAK
            size = j - i + 1
            max = max(max, size)
    RETURN max
```

 $O(n^2)$

Source Code

```
package com.basics;
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements (0s and 1s):");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        int max = 0;

        for (int i = 0; i < ar.length; i++) {
            int zeroCount = 0;
            for (int j = i; j < ar.length; j++) {
                if (ar[j] == 0) zeroCount++;
                if (zeroCount > 1) break;
                int size = j - i + 1;
                if (size > max) max = size;
            }
        }

        System.out.println("Longest sequence = " + max);
        sc.close();
    }
}
```

3.2 Optimal Sliding Window (Flip 1 bit)

Methodology

- Use two pointers (left, right)
- Expand right; count zeros
- Shrink left while zeroCount > 1
- Track maximum window size

Pseudocode

```
FUNCTION optimal(ar):  
    left = 0, zeroCount = 0, max = 0  
    FOR right = 0 TO n-1:  
        IF ar[right] == 0: zeroCount++  
        WHILE zeroCount > 1:  
            IF ar[left] == 0: zeroCount--  
            left++  
        size = right - left + 1  
        max = max(max, size)  
    RETURN max
```

 $O(n)$

Source Code

```
package com.basics;
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements (0s and 1s):");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        int left = 0;
        int zeroCount = 0;
        int max = 0;

        for (int right = 0; right < ar.length; right++) {
            if (ar[right] == 0) zeroCount++;

            while (zeroCount > 1) {
                if (ar[left] == 0) zeroCount--;
                left++;
            }

            int size = right - left + 1;
            if (size > max) max = size;
        }

        System.out.println("Longest sequence = " + max);
        sc.close();
    }
}
```

3.3 Sliding Window (Flip k bits)

Pseudocode

```
FUNCTION optimalK(ar, k):  
    left = 0, zeroCount = 0, max = 0  
    FOR right = 0 TO n-1:  
        IF ar[right] == 0: zeroCount++  
        WHILE zeroCount > k:  
            IF ar[left] == 0: zeroCount--  
            left++  
        size = right - left + 1  
        max = max(max, size)  
    RETURN max
```

 $O(n)$

Source Code

```
package com.basics;

import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements (0s and 1s):");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        System.out.print("Enter k (number of bits to flip): ");
        int k = sc.nextInt();

        int left = 0;
        int zeroCount = 0;
        int max = 0;

        for (int right = 0; right < ar.length; right++) {
            if (ar[right] == 0) zeroCount++;

            while (zeroCount > k) {
                if (ar[left] == 0) zeroCount--;
                left++;
            }

            int size = right - left + 1;
            if (size > max) max = size;
        }

        System.out.println("Longest sequence = " + max);
        sc.close();
    }
}
```

4. Swap Two Nibbles in a Byte

Methodology

- Extract left nibble using AND with 0xF0
- Extract right nibble using AND with 0x0F
- Right shift left nibble by 4, left shift right nibble by 4
- Combine using OR

Pseudocode

```
FUNCTION swapNibbles(num):  
    leftNibble = num & 0xF0  
    rightNibble = num & 0x0F  
    res = (leftNibble >> 4) | (rightNibble << 4)  
    RETURN res
```

 O(1)

Source Code

```
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter a number: ");  
        int num = sc.nextInt();  
  
        int leftNibble = num & 0xF0;  
        int rightNibble = num & 0x0F;  
  
        int res = (leftNibble >> 4) | (rightNibble << 4);  
  
        System.out.println("After swapping nibbles: " + res);  
        sc.close();  
    }  
}
```

Example: Input 53 (0011 0101) → Output 83 (0101 0011)

5. Block Swap Algorithm

5.1 Left Rotation

Methodology

- Reverse first k elements
- Reverse remaining n-k elements
- Reverse entire array

Pseudocode

```
FUNCTION reverse(arr, left, right):  
    WHILE left < right:  
        swap(arr[left], arr[right])  
        left++, right--  
  
FUNCTION leftRotate(arr, k):  
    k = k % n  
    reverse(arr, 0, k-1)  
    reverse(arr, k, n-1)  
    reverse(arr, 0, n-1)
```

 $O(n)$

Source Code

```
import java.util.Scanner;

public class Main {

    public static void reverse(int[] arr, int left, int right) {
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter number of left rotations: ");
        int k = sc.nextInt();

        k = k % n;

        reverse(arr, 0, k - 1);
        reverse(arr, k, n - 1);
        reverse(arr, 0, n - 1);

        System.out.println("Rotated array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}
```

5.2 Right Rotation

Pseudocode

```
FUNCTION rightRotate(arr, k):  
    k = k % n  
    reverse(arr, 0, n-1)  
    reverse(arr, 0, k-1)  
    reverse(arr, k, n-1)
```

 $O(n)$

Source Code

```
import java.util.Scanner;

public class Main {

    public static void reverse(int[] arr, int left, int right) {
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter number of right rotations: ");
        int k = sc.nextInt();

        k = k % n;

        reverse(arr, 0, n - 1);
        reverse(arr, 0, k - 1);
        reverse(arr, k, n - 1);

        System.out.println("Rotated array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}
```

5.3 Reverse Between Two Indexes

Source Code

```
import java.util.Scanner;

public class Main {

    static void reverse(int[] arr, int left, int right) {
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter left index: ");
        int left = sc.nextInt();

        System.out.print("Enter right index: ");
        int right = sc.nextInt();

        reverse(arr, left, right);

        System.out.println("Array after reversing between " + left + " and " + right + ":");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}
```

5.4 Reverse First k and Last k Elements

Source Code

```
import java.util.Scanner;

public class Main {

    static void reverse(int[] arr, int left, int right) {
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter k: ");
        int k = sc.nextInt();

        reverse(arr, 0, k - 1);
        reverse(arr, n - k, n - 1);

        System.out.println("Array after reversing first " + k + " and last " + k + " elements");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}
```

6. Max Product Subarray

6.1 Brute Force (Triple Loop)

Pseudocode

```
FUNCTION bruteForceTriple(ar):  
    max = -INF  
    FOR i = 0 TO n-1:  
        FOR j = i TO n-1:  
            prod = 1  
            FOR k = i TO j:  
                prod *= ar[k]  
            max = max(max, prod)  
    RETURN max
```

 $O(n^3)$

Source Code

```
package com.basics;

import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        int max = Integer.MIN_VALUE;

        for (int i = 0; i < ar.length; i++) {
            for (int j = i; j < ar.length; j++) {
                int prod = 1;
                for (int k = i; k <= j; k++) {
                    prod = prod * ar[k];
                }
                if (prod > max) max = prod;
            }
        }

        System.out.println("Maximum product = " + max);
        sc.close();
    }
}
```

6.2 Brute Force (Double Loop)

Pseudocode

```
FUNCTION bruteForceDouble(ar):  
    max = -INF  
    FOR i = 0 TO n-1:  
        prod = 1  
        FOR j = i TO n-1:  
            prod *= ar[j]  
            max = max(max, prod)  
    RETURN max
```

 $O(n^2)$

Source Code

```
package com.basics;  
import java.util.Scanner;  
  
class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter array size: ");  
        int n = sc.nextInt();  
  
        int[] ar = new int[n];  
        System.out.println("Enter array elements:");  
        for (int i = 0; i < n; i++) {  
            ar[i] = sc.nextInt();  
        }  
  
        int max = Integer.MIN_VALUE;  
  
        for (int i = 0; i < ar.length; i++) {  
            int prod = 1;  
            for (int j = i; j < ar.length; j++) {  
                prod = prod * ar[j];  
                if (prod > max) max = prod;  
            }  
        }  
  
        System.out.println("Maximum product = " + max);  
        sc.close();  
    }  
}
```

6.3 Optimal (Left-Right Product)

Methodology

- Traverse left to right, keeping running product
- Traverse right to left, keeping running product
- Reset product to 1 when it becomes 0
- Track maximum of both passes

Pseudocode

```
FUNCTION optimal(ar):  
    max = -INF  
    leftProd = 1, rightProd = 1  
    FOR i = 0 TO n-1:  
        leftProd *= ar[i]  
        max = max(max, leftProd)  
        IF leftProd == 0: leftProd = 1  
  
        rightProd *= ar[n-1-i]  
        max = max(max, rightProd)  
        IF rightProd == 0: rightProd = 1  
    RETURN max
```

 $O(n)$

Source Code

```
package com.basics;

import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        int max = Integer.MIN_VALUE;
        int leftProd = 1;
        int rightProd = 1;

        for (int i = 0; i < ar.length; i++) {
            leftProd = leftProd * ar[i];
            if (leftProd > max) max = leftProd;
            if (leftProd == 0) leftProd = 1;

            rightProd = rightProd * ar[ar.length - 1 - i];
            if (rightProd > max) max = rightProd;
            if (rightProd == 0) rightProd = 1;
        }

        System.out.println("Maximum product = " + max);
        sc.close();
    }
}
```

7. Maximum Sum of Hourglass in Matrix

Methodology

- Hourglass needs 3×3 area

- Pattern: top row (3), middle center (1), bottom row (3)
- Slide window across matrix
- Track maximum sum

Pseudocode

```
FUNCTION maxHourglass(ar):  
    IF rows < 3 OR cols < 3: RETURN "Cannot form"  
    max = -INF  
    FOR i = 0 TO rows-3:  
        FOR j = 0 TO cols-3:  
            sum = ar[i][j] + ar[i][j+1] + ar[i][j+2]  
                  + ar[i+1][j+1]  
                  + ar[i+2][j] + ar[i+2][j+1] + ar[i+2][j+2]  
            max = max(max, sum)  
    RETURN max
```

 $O(\text{rows} \times \text{cols})$

Source Code

```
package com.basics;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of rows: ");
        int row = sc.nextInt();

        System.out.print("Enter number of columns: ");
        int col = sc.nextInt();

        int[][] ar = new int[row][col];
        System.out.println("Enter matrix elements:");
        for (int i = 0; i < row; i++) {
            for (int j = 0; j < col; j++) {
                ar[i][j] = sc.nextInt();
            }
        }

        if (row < 3 || col < 3) {
            System.out.println("Hourglass cannot be formed");
            sc.close();
            return;
        }

        int max = Integer.MIN_VALUE;

        for (int i = 0; i <= row - 3; i++) {
            for (int j = 0; j <= col - 3; j++) {
                int sum = ar[i][j] + ar[i][j + 1] + ar[i][j + 2]
                    + ar[i + 1][j + 1]
                    + ar[i + 2][j] + ar[i + 2][j + 1] + ar[i + 2][j + 2];

                if (sum > max) max = sum;
            }
        }

        System.out.println("Maximum Hourglass Sum: " + max);
        sc.close();
    }
}
```

8. Max Equilibrium Sum

8.1 Brute Force

Pseudocode

```
FUNCTION bruteForce(ar):  
    max = -INF  
    FOR i = 0 TO n-1:  
        leftSum = 0, rightSum = 0  
        FOR j = 0 TO i-1: leftSum += ar[j]  
        FOR j = i+1 TO n-1: rightSum += ar[j]  
        IF leftSum == rightSum:  
            max = max(max, leftSum)  
    RETURN max
```

 $O(n^2)$

Source Code

```
package com.basics;

import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        int max = Integer.MIN_VALUE;

        for (int i = 0; i < ar.length; i++) {
            int leftSum = 0;
            int rightSum = 0;

            for (int j = 0; j < i; j++) leftSum += ar[j];
            for (int j = i + 1; j < ar.length; j++) rightSum += ar[j];

            if (leftSum == rightSum) {
                if (leftSum > max) max = leftSum;
            }
        }

        if (max == Integer.MIN_VALUE) {
            System.out.println("No equilibrium index");
        } else {
            System.out.println("Maximum equilibrium sum: " + max);
        }
        sc.close();
    }
}
```

8.2 Prefix-Suffix Arrays

Methodology

- Build prefix sum array

- Build suffix sum array
- $\text{leftSum} = \text{prefix}[i] - \text{ar}[i]$
- $\text{rightSum} = \text{suffix}[i] - \text{ar}[i]$

Pseudocode

```
FUNCTION prefixSuffix(ar):
    prefix[0] = ar[0]
    FOR i = 1 TO n-1: prefix[i] = prefix[i-1] + ar[i]

    suffix[n-1] = ar[n-1]
    FOR i = n-2 DOWNT0 0: suffix[i] = suffix[i+1] + ar[i]

    max = -INF
    FOR i = 0 TO n-1:
        leftSum = prefix[i] - ar[i]
        rightSum = suffix[i] - ar[i]
        IF leftSum == rightSum:
            max = max(max, leftSum)
    RETURN max
```

 $O(n)$ |  $O(n)$

Source Code

```
package com.basics;

import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        int[] prefix = new int[ar.length];
        int sum = 0;
        for (int i = 0; i < ar.length; i++) {
            sum += ar[i];
            prefix[i] = sum;
        }

        sum = 0;
        int[] suffix = new int[ar.length];
        for (int i = ar.length - 1; i >= 0; i--) {
            sum += ar[i];
            suffix[i] = sum;
        }

        int max = Integer.MIN_VALUE;
        for (int i = 0; i < ar.length; i++) {
            int leftSum = prefix[i] - ar[i];
            int rightSum = suffix[i] - ar[i];
            if (leftSum == rightSum) {
                if (leftSum > max) max = leftSum;
            }
        }

        if (max == Integer.MIN_VALUE) {
            System.out.println("No equilibrium index");
        } else {
            System.out.println("Maximum equilibrium sum: " + max);
        }
        sc.close();
    }
}
```

```
}  
}
```

8.3 Most Optimal (Single Pass)

Methodology

- Compute total sum
- Maintain running left sum
- $\text{rightSum} = \text{totalSum} - \text{leftSum} - \text{ar}[i]$

Pseudocode

```
FUNCTION mostOptimal(ar):  
    totalSum = sum(ar)  
    leftSum = 0  
    max = -INF  
    FOR i = 0 TO n-1:  
        rightSum = totalSum - leftSum - ar[i]  
        IF leftSum == rightSum:  
            max = max(max, leftSum)  
        leftSum += ar[i]  
    RETURN max
```

 $O(n)$ |  $O(1)$

Source Code

```
package com.basics;
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        int totalSum = 0;
        for (int i = 0; i < ar.length; i++) totalSum += ar[i];

        int leftSum = 0;
        int max = Integer.MIN_VALUE;

        for (int i = 0; i < ar.length; i++) {
            int rightSum = totalSum - leftSum - ar[i];

            if (leftSum == rightSum) {
                max = Math.max(max, leftSum);
            }

            leftSum += ar[i];
        }

        if (max == Integer.MIN_VALUE) {
            System.out.println("No equilibrium index");
        } else {
            System.out.println("Maximum equilibrium sum: " + max);
        }
        sc.close();
    }
}
```

9. Leaders in Array

Definition: An element is a leader if it is greater than or equal to all elements on its right.

9.1 Brute Force

Pseudocode

```
FUNCTION isLeader(element, ind, arr):  
    FOR i = ind TO n-1:  
        IF element < arr[i]: RETURN false  
    RETURN true  
  
FUNCTION bruteForce(ar):  
    list = []  
    FOR i = 0 TO n-1:  
        IF isLeader(ar[i], i+1, ar):  
            list.addFirst(ar[i])  
    RETURN list
```

 $O(n^2)$

Source Code

```
package com.basics;

import java.util.LinkedList;
import java.util.Scanner;

public class Main {

    public static boolean isLeader(int element, int ind, int[] arr) {
        for (int i = ind; i < arr.length; i++) {
            if (element < arr[i]) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        LinkedList list = new LinkedList();

        for (int i = 0; i < ar.length; i++) {
            if (isLeader(ar[i], i + 1, ar)) {
                list.addFirst(ar[i]);
            }
        }

        System.out.println("Leaders:");
        for (int x : list) System.out.print(x + " ");
        sc.close();
    }
}
```

9.2 Optimal (Right to Left)

Methodology

- Traverse from right

- Track maximum seen so far
- If current $\geq$ max, it's a leader

Pseudocode

```
FUNCTION optimal(ar):  
    list = []  
    max = -INF  
    FOR i = n-1 DOWNT0 0:  
        IF ar[i]  $\geq$  max:  
            list.addFirst(ar[i])  
            max = ar[i]  
    RETURN list
```

 $O(n)$

Source Code

```
package com.basics;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] ar = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            ar[i] = sc.nextInt();
        }

        LinkedList list = new LinkedList();
        int max = Integer.MIN_VALUE;

        for (int i = ar.length - 1; i >= 0; i--) {
            if (ar[i] >= max) {
                list.addFirst(ar[i]);
                max = ar[i];
            }
        }

        System.out.println("Leaders:");
        for (int x : list) System.out.print(x + " ");
        sc.close();
    }
}
```

10. Majority Element

Methodology

- Count frequency of each element using HashMap
- Element with frequency $> n/2$ is majority

Pseudocode

```
FUNCTION majorityElement(arr):  
    map = {}  
    FOR each num in arr:  
        map[num] = map.getOrDefault(num, 0) + 1  
    FOR each entry in map:  
        IF entry.value > n/2:  
            RETURN entry.key  
    RETURN "No majority"
```

 $O(n)$ |  $O(n)$

Source Code

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        HashMap map = new HashMap<>();

        for (int i = 0; i < arr.length; i++) {
            map.put(arr[i], map.getOrDefault(arr[i], 0) + 1);
        }

        boolean found = false;
        for (Map.Entry entry : map.entrySet()) {
            if (entry.getValue() > arr.length / 2) {
                System.out.println("Majority Element: " + entry.getKey());
                found = true;
                break;
            }
        }

        if (!found) System.out.println("No majority element");
        sc.close();
    }
}
```

11. Lexicographically First Palindromic String

Methodology

- Count character frequencies using TreeMap (sorted)
- If more than 1 odd frequency → invalid
- Build left half from sorted characters
- Middle = odd character (if any)
- Right half = reverse of left

Pseudocode

```
FUNCTION firstPalindrome(s):
    map = TreeMap()
    FOR each ch in s:
        map[ch] = map.getOrDefault(ch, 0) + 1

    oddCount = 0, oddChar = ' '
    FOR each entry in map:
        IF entry.value % 2 != 0:
            oddCount++
            oddChar = entry.key

    IF oddCount > 1: RETURN "Invalid"

    sb = StringBuilder()
    FOR each entry in map:
        FOR i = 0 TO freq/2 - 1:
            sb.append(ch)

    left = sb.toString()
    right = sb.reverse().toString()

    IF oddCount == 1:
        RETURN left + oddChar + right
    ELSE:
        RETURN left + right
```

 $O(n \log n)$

Source Code

```
package com.basics;

import java.util.Map;
import java.util.Scanner;
import java.util.TreeMap;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter string: ");
        String s = sc.next();

        TreeMap map = new TreeMap();

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        int oddCount = 0;
        char oddChar = ' ';

        for (Map.Entry entry : map.entrySet()) {
            if (entry.getValue() % 2 != 0) {
                oddCount++;
                oddChar = entry.getKey();
            }
        }

        if (oddCount > 1) {
            System.out.println("Invalid String");
            sc.close();
            return;
        }

        StringBuilder sb = new StringBuilder();

        for (Map.Entry entry : map.entrySet()) {
            int freq = entry.getValue();
            char ch = entry.getKey();
            for (int i = 0; i < freq / 2; i++) {
                sb.append(ch);
            }
        }

        String left = sb.toString();
        String right = sb.reverse().toString();
    }
}
```

```
String res = "";

if (oddCount == 1) res = left + oddChar + right;
else res = left + right;

System.out.println("Lexicographically first palindrome: " + res);
sc.close();
}
}
```

12. Natural Sort Order

Methodology

- Split strings into text and number parts using regex
- Compare text parts lexicographically
- Compare number parts numerically
- Continue until difference found

Pseudocode

```
FUNCTION naturalCompare(s1, s2):  
    pattern = /(\D*)(\d*)/  
    m1 = pattern.matcher(s1)  
    m2 = pattern.matcher(s2)  
  
    WHILE m1.find() AND m2.find():  
        textCompare = m1.group(1).compareTo(m2.group(1))  
        IF textCompare != 0: RETURN textCompare  
  
        num1 = m1.group(2), num2 = m2.group(2)  
        IF num1.isEmpty() AND num2.isEmpty(): CONTINUE  
  
        n1 = num1.isEmpty() ? 0 : parseInt(num1)  
        n2 = num2.isEmpty() ? 0 : parseInt(num2)  
        IF n1 != n2: RETURN n1 - n2  
  
    RETURN s1.compareTo(s2)
```

 $O(n \log n \times m)$

Source Code

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

class NaturalOrderComparator implements Comparator {
    Pattern pattern = Pattern.compile("(\\D*)(\\d*)");

    public int compare(String s1, String s2) {
        Matcher m1 = pattern.matcher(s1);
        Matcher m2 = pattern.matcher(s2);

        while (m1.find() && m2.find()) {
            int textCompare = m1.group(1).compareTo(m2.group(1));
            if (textCompare != 0) return textCompare;

            String num1 = m1.group(2);
            String num2 = m2.group(2);

            if (num1.isEmpty() && num2.isEmpty()) continue;

            int n1 = num1.isEmpty() ? 0 : Integer.parseInt(num1);
            int n2 = num2.isEmpty() ? 0 : Integer.parseInt(num2);

            if (n1 != n2) return n1 - n2;
        }
        return s1.compareTo(s2);
    }
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter number of strings: ");
        int n = scanner.nextInt();
        scanner.nextLine();

        List list = new ArrayList<>();
        System.out.println("Enter the strings:");
        for (int i = 0; i < n; i++) {
            list.add(scanner.nextLine());
        }
    }
}
```

```
Collections.sort(list, new NaturalOrderComparator());

System.out.println("Natural Sorted Order:");
for (String s : list) {
    System.out.println(s);
}
scanner.close();
}
```

Summary Table

#	Algorithm	Best Time Complexity	Space Complexity
1	HCF - Brute Force	$O(\min(a,b))$	$O(1)$
2	HCF - Subtraction	$O(\max(a,b))$	$O(1)$
3	HCF - Euclidean	$O(\log(\min(a,b)))$	$O(1)$
4	Karatsuba	$O(n^{1.585})$	$O(\log n)$
5	Longest 1s - Brute	$O(n^2)$	$O(1)$
6	Longest 1s - Optimal	$O(n)$	$O(1)$
7	Swap Nibbles	$O(1)$	$O(1)$
8	Left Rotation	$O(n)$	$O(1)$
9	Right Rotation	$O(n)$	$O(1)$
10	Max Product - Brute	$O(n^2)$	$O(1)$
11	Max Product - Optimal	$O(n)$	$O(1)$
12	Hourglass Sum	$O(\text{rows} \times \text{cols})$	$O(1)$

#	Algorithm	Best Time Complexity	Space Complexity
13	Equilibrium - Brute	$O(n^2)$	$O(1)$
14	Equilibrium - Prefix	$O(n)$	$O(n)$
15	Equilibrium - Optimal	$O(n)$	$O(1)$
16	Leaders - Brute	$O(n^2)$	$O(1)$
17	Leaders - Optimal	$O(n)$	$O(1)$
18	Majority Element	$O(n)$	$O(n)$
19	First Palindrome	$O(n \log n)$	$O(n)$
20	Natural Sort	$O(n \log n \times m)$	$O(n)$

Tip: Press **Ctrl + P** (or **Cmd + P** on Mac) and select **"Save as PDF"** to export this entire document as a PDF file!